PORTFOLIO CASE STUDY // 2026

Cartridge & Console

A full-stack second-hand console & game marketplace, built solo and shipped live across three platforms.

REACT | ASP.NET CORE | MYSQL | JWT AUTH | TAILWIND

● LIVE

Mongezi "Mo" Mabuza, Full-Stack Developer
github.com/hue-we

01 — OVERVIEW

A marketplace, not a template

Cartridge & Console is a CRUD marketplace app where people buy and sell second-hand consoles, cartridges, and discs directly with each other. The brief was simple enough on paper: full CRUD, JWT authentication, a relational backend. The real goal was to build something that looked and felt like a real product instead of a scaffolded demo.

The build covers the full stack end to end. A React front end, an ASP.NET Core Web API, a MySQL relational database, and stateless JWT-based authentication with per-user authorization checked on every write.

ROLE	**Solo full-stack developer**
STACK	React, Vite, Tailwind CSS · ASP.NET Core 8 · MySQL (Pomelo/EF Core) · JWT
SCOPE	Data modelling, REST API, auth, UI/UX design, containerized deployment
HOSTING	Netlify (frontend) · Render (API, Docker) · Railway (MySQL)

02 — THE PROBLEM

No middleman, no markup

Second-hand console and game sellers don't really have a lightweight place to list their stock. Buyers don't have a central place to browse listings that are priced and described consistently either. Most of it happens in scattered marketplace posts with no real structure to compare against.

Requirements defined up front:

→ Anyone can browse, search, and filter listings without an account.

→ Registered sellers can create, edit, and delete only their own listings.

→ Passwords are never stored in plaintext.

→ The API stays stateless and scalable, with no server-side sessions to manage.

→ Clear separation of concerns: API, database, and client each own their layer.

03 — DESIGN DIRECTION

Thrift-shop, not template

The brief pointed to a generic purple SaaS-dashboard mockup as a starting point. Instead of cloning it, the design leaned into what the product actually is: a second-hand marketplace. Its visual language comes from real thrift-store price tags and old console UI text, not another gradient dashboard.

Palette

Signature element

Every listing card carries a rotated paper **price tag** component, complete with a punched hole, sitting on the product image the way a real second-hand price sticker would. It's a small detail, but it's the one piece of UI that makes the app feel like a thrift shop instead of a spreadsheet.

Type system

Bold display type for headings, a clean sans body face for readability, and a monospace face reserved specifically for prices, statuses, and platform labels. It echoes old console UI and price-tag typography without trying too hard to look retro.

04 — ARCHITECTURE

How the pieces talk

The client, API, and database are fully decoupled, and they now live on three separate platforms talking to each other over the open internet. The React app never touches the database directly, and the API never trusts anything from the client without verifying the JWT and re-checking ownership on the server.

Netlify	Hosts the built React SPA. Pulls straight from GitHub, rebuilds on push.
Render	Runs the ASP.NET Core API from a Dockerfile, applies pending EF Core migrations automatically on startup
Railway	Hosts the MySQL database, reachable through its public proxy address

Splitting hosting across three providers instead of one all-in-one platform was a deliberate choice: it keeps each piece replaceable on its own, and it's closer to how a real small team would divide infrastructure than a single bundled deploy button.

Security decisions

→ Passwords hashed with BCrypt, never stored or logged in plaintext.

→ JWTs signed with HMAC-SHA256, validated on issuer, audience, lifetime, and signing key.

→ Every write endpoint re-checks that the JWT's user ID matches the listing's SellerId. A valid token alone isn't enough to edit someone else's listing.

→ CORS locked to the known frontend origin instead of left wide open.

05 — PROCESS, PART ONE

Getting it running locally

The code was the easy part, honestly. Getting a clean local environment working end to end took longer than writing the CRUD logic itself, which felt worth documenting since it's the part of full-stack work that tutorials tend to skip over.

NuGet restore instability

Package restores kept timing out mid-download for no obvious reason. Working through it step by step (testing the same URLs directly in a browser, clearing the NuGet cache, disabling parallel downloads) narrowed it down to something specific to the dotnet CLI rather than the network itself. Retries eventually pushed every package through.

Local MySQL access

A forgotten root password on the local MySQL install blocked every connection attempt. Instead of fighting through a password reset, switching to XAMPP's bundled MySQL server (no root password by default) got a clean local database running within minutes, with phpMyAdmin on hand for quick inspection.

06 — PROCESS, PART TWO

Taking it live

Deploying across three separate platforms surfaced a different category of problem, less about the code itself, more about how services find and trust each other over the internet. Every one of these turned out to have a specific, findable cause.

A stuck queue and a switch to Render

Railway's build queue stalled for over 20 minutes on an upstream GitHub connectivity issue, entirely outside the project itself. Rather than wait it out, the API moved to Render instead, keeping Railway for what it was already doing well: hosting the database.

Docker paths, twice

Render builds the API from a Dockerfile, which meant getting the root directory, build context, and Dockerfile path all pointed at the same folder. The first attempt doubled up a folder name in the path, the second had the Dockerfile sitting one directory too high. Both were one-line fixes once the exact path Render was resolving got printed in the build log.

A CORS error that wasn't really a CORS error

The browser reported a CORS block on every request from the live frontend to the live API. The actual `Cors:AllowedOrigin` config was correct the whole time. The real issue: a server throwing a 500 doesn't send CORS headers back at all, so the browser reports it as a CORS failure regardless of the real cause. That's a distinction worth remembering, a CORS error in the console can be a symptom of something else entirely.

The actual cause: an unrun migration

Underneath the CORS red herring was a much simpler problem: the EF Core migration had never actually been run against the live Railway database, so the Users and Listings tables didn't exist yet. Running the migration from a local machine failed too, this time from local network flakiness reaching Railway's proxy port. The fix that stuck was having the API apply any pending migrations automatically the moment it starts up, using the working connection it already has from inside Render.

07 — WHAT'S NEXT

It's live. Now what.

With all three platforms talking to each other correctly, the marketplace is live and serving real requests end to end. From here, it's less about infrastructure and more about the product itself.

Coming up

→ Seed the live database with real listings and real images, so the link doesn't land on an empty page.

→ A custom domain instead of the default Netlify/Render subdomains.

→ Rotate the credentials that were exposed while debugging deployment.

→ A README with setup instructions and a screenshot for anyone browsing the repo.

Still out of scope for v1

→ Payments/escrow between buyer and seller

→ In-app messaging

→ Image upload storage (currently image URL only)

→ Admin moderation tooling

Mongezi Mabuza · Full-Stack Developer, Johannesburg/Pretoria
github.com/hue-we · linkedin.com/in/mongezi-mabuza-7198172b0